TRIP PLANNER PWA · FICHE MODULE · MEMBRE 1

Module 1 — Gestion des utilisateurs

Ce que fait le module, la logique métier qu'il porte, et ce que vous devez savoir présenter.

COLLECTION MONGODB	FONCTIONNALITÉ AVANCÉE	RÔLE TRANSVERSE
users	**Export PDF de l'itinéraire et liens de partage publics**	**Authentification et mise en page commune (AppShell)**

1. Objectif du module

Gérer le cycle de vie complet d'un compte : inscription, connexion, session sécurisée, mot de passe oublié, profil, suppression du compte et administration (rôles, désactivation, statistiques). Le module fournit aussi le middleware d'authentification utilisé par tous les autres modules, ainsi que deux fonctions de diffusion : l'export PDF d'un voyage et le lien public en lecture seule.

> Règle d'équipe : vous êtes le seul à écrire dans votre collection. Les autres modules passent par votre API (ou votre service), et toute modification d'un modèle partagé demande l'accord du groupe.

2. Modèle de données

Champ	Type	Règle / rôle
`name`	texte	2 à 80 caractères
`email`	texte	unique, stocké en minuscules
`passwordHash`	texte	bcrypt, coût 12 — jamais renvoyé par l'API
`avatarUrl`	texte | null	URL Cloudinary ou /uploads
`baseCurrency`	code ISO 4217	devise par défaut proposée à la création d'un voyage (EUR)
`language`	texte	préférence enregistrée (interface en anglais uniquement, hors périmètre)
`homeCountry`	code ISO 3166	sert à la règle « destination ≠ pays de résidence » de la checklist
`role`	member | admin	contrôle l'accès aux routes d'administration
`isActive`	booléen	false = compte désactivé par un administrateur
`pushSubscriptions[]`	objets	un abonnement Web Push par appareil
`notificationPrefs`	objet	forumReplies (on/off) + mutedPosts (fils mis en sourdine)
`refreshTokens[]`	objets	jti, expiresAt, rotatedAt — sessions ouvertes
`resetTokenHash / resetTokenExpires`	texte / date	jeton de réinitialisation haché (SHA-256), valable 1 h
`lastLoginAt, createdAt, updatedAt`	dates	statistiques et audit
`lastLoginIp`	texte | null	adresse IP de la dernière connexion (visible par l'utilisateur et l'admin)

Champ	Type	Règle / rôle
`loginHistory[]`	objets	10 dernières connexions : { ip, device, method: login | register, at }, la plus récente en premier

3. Logique métier

Inscription et connexion

- Le corps de chaque requête est validé avec **zod** : nom de 2 à 80 caractères, email valide, mot de passe d'au moins 8 caractères.
- Un email déjà utilisé renvoie **409** avec une erreur sur le champ `email`.
- Le mot de passe est haché avec **bcrypt (coût 12)** ; le hash n'est jamais renvoyé (filtré par `toJSON`).
- En cas d'échec de connexion, le message est volontairement générique (« Invalid email or password ») pour ne pas révéler si le compte existe.
- Un compte désactivé ne peut pas se connecter (**403**).
- Les routes d'authentification sont limitées à **20 tentatives / 15 min / IP** (réponse 429).

Captcha (protection contre les robots)

- Captcha **auto-hébergé** : l'API génère l'image (*svg-captcha*) ; aucun service externe ni clé n'est nécessaire, et le navigateur ne contacte que notre API (règle du cahier des charges).
- `GET /auth/captcha` renvoie un identifiant et une image SVG de **5 caractères** dessinés en tracés (pas en texte lisible dans le code), sans caractères ambigus (0/O, 1/l/I). Limité à 30 images par minute et par IP.
- Le serveur ne garde que le **hash SHA-256** de la réponse, **5 minutes** au maximum. La comparaison ignore la casse et se fait en temps constant.
- Chaque captcha est **à usage unique** : il est supprimé après une tentative, bonne ou mauvaise. Le formulaire charge alors une nouvelle image.
- Obligatoire à l'**inscription** et pour le **mot de passe oublié**. Réponse fausse ou absente → 400, erreur sur le champ `captcha` et `captchaRequired: true`.
- **Connexion** : après **3 échecs en 15 minutes** pour le même email, la connexion exige un captcha, même avec le bon mot de passe. Le compteur est lié à l'**email** (pas à l'IP), donc changer ou falsifier son IP ne permet pas de l'éviter ; une connexion réussie le remet à zéro.
- Désactivable avec `CAPTCHA_ENABLED=false` (tests automatisés). Limite : les captchas en attente sont gardés en mémoire, donc avec une seule instance de l'API.

Adresse IP et historique des connexions

- À chaque **connexion** ou **inscription** réussie, l'API enregistre l'**adresse IP**, l'appareil (navigateur et système, déduits du User-Agent) et la date.
- Les **10 dernières** connexions sont conservées (`loginHistory`, la plus récente en premier) ; `lastLoginIp` garde la dernière.
- Le rafraîchissement du jeton n'est pas une nouvelle connexion : il n'est pas enregistré.
- L'utilisateur voit ses connexions dans son profil (« Recent connections ») et est invité à changer son mot de passe s'il ne reconnaît pas l'une d'elles. L'administrateur voit la dernière IP de chaque compte.
- L'IP est lue dans l'en-tête `X-Forwarded-For` selon `TRUST_PROXY`. Le proxy Next.js n'ajoute pas d'entrée : en production, un reverse proxy (nginx, hébergeur) placé devant Next fournit l'IP réelle ; en local elle vaut 127.0.0.1. Une valeur qui n'est pas une IP valide est ignorée.
- Donnée **informative** : aucune décision de sécurité ne repose sur l'IP, puisqu'un en-tête peut être falsifié sans reverse proxy.

Session : access token + refresh token avec rotation

- **Access token** JWT valable **15 minutes**, contenant l'id et le rôle, envoyé dans l'en-tête `Authorization: Bearer`. Côté client il reste en mémoire (pas dans localStorage) pour limiter le risque XSS.
- **Refresh token** JWT valable **7 jours**, stocké dans un cookie **httpOnly** limité au chemin `/api/v1/auth`. Son identifiant unique (`jti`) est enregistré sur l'utilisateur.
- **Rotation à chaque usage** : `POST /auth/refresh` marque l'ancien jti comme « tourné » et en émet un nouveau.
- **Détection de vol** : si un jeton déjà tourné est présenté plus de 15 s après sa rotation, toutes les sessions de l'utilisateur sont révoquées.
- Une **fenêtre de grâce de 15 s** évite de déconnecter l'utilisateur quand deux onglets (ou le service worker) rafraîchissent en même temps.
- Le middleware `requireAuth` vérifie le JWT puis relit `role` et `isActive` en base : une désactivation ou un changement de rôle prend effet immédiatement.
- Le client fait un rafraîchissement « single-flight » (une seule requête partagée) et rejoue automatiquement une requête refusée en 401.

Mot de passe oublié

- La réponse est identique que le compte existe ou non (protection contre l'énumération d'emails).
- Le jeton est une valeur aléatoire de **32 octets** ; seul son **hash SHA-256** est stocké, avec une expiration d'**1 heure**.
- Le lien est envoyé par SMTP si configuré, sinon affiché dans la console de l'API (développement).
- La réinitialisation invalide toutes les sessions ouvertes (refreshTokens vidés).
- Le changement de mot de passe depuis le profil exige l'ancien mot de passe.

Suppression du compte (règle 2 du modèle de données)

- Le mot de passe est redemandé.
- Dans **une transaction** : les voyages dont il est propriétaire sont supprimés en cascade, il est retiré des voyages des autres, ses posts et réponses du forum sont **anonymisés** (auteur = null, affiché « Deleted user »), il est retiré des abonnés des fils, puis le compte est supprimé.
- Les fils de discussion restent donc lisibles pour la communauté.

Administration

- Liste paginée des comptes avec recherche par nom ou email.
- Désactiver / réactiver un compte : la désactivation ferme aussi toutes ses sessions.
- Passer un membre administrateur (ou l'inverse). Un administrateur ne peut pas modifier son propre compte depuis cet écran.
- Statistiques : nombre d'utilisateurs, actifs et nouveaux sur 7 jours, voyages, lieux, dépenses, articles de checklist, entrées de journal, posts, réponses, signalements ouverts, top 5 des pays de destination.

4. Fonctionnalité avancée — Export PDF de l'itinéraire et liens de partage publics

Export PDF — `GET /trips/:id/export` (membre du voyage) : le PDF est généré **côté serveur** avec *pdfkit* et envoyé en flux. Il contient l'en-tête (titre, destination, dates, nombre de jours, budget, participants), puis chaque jour avec ses lieux triés par ordre (heure, nom, adresse, catégorie, durée, coût estimé, notes), une section « non planifiés » et la date de génération.

Lien de partage — `POST /trips/:id/share` (propriétaire uniquement) crée un `shareToken` aléatoire de **32 octets** (64 caractères hexadécimaux), avec un index unique « sparse ». L'appel est idempotent : s'il existe déjà, le même lien est renvoyé.

`DELETE /trips/:id/share` **révoque** le lien : l'ancienne URL renvoie 404.

`GET /share/:token` est **public** : le format du jeton est vérifié, puis seules les informations non sensibles sont renvoyées (titre, destination, dates, couverture, prénoms des participants, lieux). Les dépenses et le journal restent privés.

5. API REST (préfixe `/api/v1`, réponse `{ data, message, errors }`)

Méthode	Route	Accès	Rôle
GET	`/auth/captcha`	public	nouveau captcha (id + image SVG)
POST	`/auth/register`	public + captcha	créer un compte et ouvrir la session
POST	`/auth/login`	public (captcha après 3 échecs)	connexion + enregistrement IP
POST	`/auth/refresh`	cookie	nouveau access token + rotation
POST	`/auth/logout`	cookie	fermer la session
POST	`/auth/forgot-password`	public + captcha	envoyer le lien de réinitialisation
POST	`/auth/reset-password`	public	choisir un nouveau mot de passe
GET / PATCH / DELETE	`/users/me`	membre	profil, modification, suppression
POST	`/users/me/avatar`	membre	photo de profil (multipart)
PATCH	`/users/me/password`	membre	changer le mot de passe
PATCH	`/users/me/notifications`	membre	activer/désactiver les notifications du forum
POST / DELETE	`/users/me/push-subscription`	membre	abonnement Web Push de l'appareil
GET	`/push/public-key`	public	clé VAPID publique
GET	`/admin/users`	admin	liste et recherche
PATCH	`/admin/users/:id`	admin	désactiver / changer le rôle
GET	`/admin/stats`	admin	statistiques d'usage
GET	`/trips/:id/export`	membre du voyage	PDF de l'itinéraire
POST / DELETE	`/trips/:id/share`	propriétaire	créer / révoquer le lien public
GET	`/share/:token`	public	itinéraire en lecture seule

6. Écrans (front Next.js)

- Connexion, inscription, mot de passe oublié, réinitialisation
- Profil et paramètres : photo, pays, devise, langue, notifications, mot de passe, suppression du compte
- Page publique `/share/[token]` : carte et programme jour par jour

- Administration : onglets *Statistics* et *Users*
- En-tête commun, menu du compte, navigation mobile, protection des pages (`RequireAuth`)

7. Fichiers dont vous êtes responsable

API — trip-planner-api

- `src/modules/users/ (user.model, auth.routes, user.routes, admin.routes, captcha.service)`
- `src/lib/clientInfo.ts (adresse IP, appareil)`
- `src/lib/tokens.ts, src/lib/mailer.ts`
- `src/middleware/auth.ts, src/middleware/rateLimit.ts`
- `src/modules/trips/export.pdf.ts + routes /share et /export de trip.routes.ts`

Front — trip-planner-web

- `src/app/(auth)/*, src/app/profile, src/app/share/[token]`
- `src/app/admin (onglets Statistics et Users)`
- `src/lib/auth.tsx, src/lib/api.ts`
- `src/components/AppShell.tsx, src/components/Captcha.tsx`

8. Liens avec les autres modules

- **Tous les modules** utilisent `requireAuth` et la session définis ici.
- **Module 2** : la suppression d'un compte appelle la suppression en cascade des voyages ; les routes de partage et d'export sont rangées dans le fichier des voyages (à coordonner).
- **Module 5** : `homeCountry` alimente la règle « à l'étranger » de la checklist.
- **Module 7** : abonnements push et préférences de notification ; anonymisation des posts.

9. À démontrer à la soutenance

- ☐ Créer un compte en résolvant le captcha ; se déconnecter, se reconnecter ; recharger la page : la session est conservée.
- ☐ Se tromper 3 fois de mot de passe : le captcha apparaît. Montrer ensuite « Recent connections » dans le profil et la dernière IP dans Admin → Users.
- ☐ Mot de passe oublié : montrer le lien dans la console de l'API, puis changer le mot de passe.
- ☐ Exporter le PDF d'un voyage ; créer le lien de partage, l'ouvrir en navigation privée, le révoquer.
- ☐ Admin : statistiques, désactiver un compte puis montrer qu'il ne peut plus se connecter.
- ☐ Supprimer un compte et montrer ses posts devenus « Deleted user ».

10. Reste à faire

- Tests d'intégration des routes d'authentification (Supertest).
- Requêtes Postman pour chaque endpoint du module.
- Configurer un vrai SMTP pour la démo (ou l'expliquer).
- Option : déplacer les routes /share et /export dans un fichier dédié du module 1.

11. Questions probables du jury

Pourquoi deux jetons au lieu d'un seul ?
Le jeton d'accès court (15 min) limite l'impact d'un vol ; le refresh en cookie httpOnly n'est pas lisible par JavaScript et sa rotation permet de détecter une réutilisation.

Pourquoi relire l'utilisateur à chaque requête ?
Pour qu'une désactivation par l'administrateur soit immédiate, sans attendre l'expiration du JWT.

Pourquoi un captcha auto-hébergé plutôt que reCAPTCHA ?
Aucun compte ni clé, fonctionne en local et sans service tiers (le client ne contacte que notre API) ; reCAPTCHA ou Turnstile resteraient possibles en production.

Pourquoi le compteur d'échecs est-il lié à l'email et pas à l'IP ?
Une IP peut être changée ou falsifiée ; lier le compteur à l'email empêche de contourner le captcha.

Pourquoi stocker le hash du jeton de réinitialisation ?
Si la base fuit, les jetons ne sont pas utilisables directement.